Lab 6: Code and Branch Coverage Analysis

Student: PES1UG24CS900
Application: L6_L7_59 — Phone Repair Quote

Objective and method

Measure statement and branch coverage, identify untested decisions, and add separate test suites that verify normal behavior, invalid inputs and boundaries. The supplied application was kept unchanged. Python coverage was restricted to phone_repair_quote.py; test files are not included in the coverage denominator. No statements or branches were excluded.

Results

Suite	Passing tests	Statements	Branch outcomes
Starter	2	18/40 (45%)	8/28 (28.57%)
Code coverage	14	40/40 (100%)	Not measured in statement-only run
Branch coverage	115	40/40 (100%)	28/28 (100%)
All suites	131	40/40 (100%)	28/28 (100%)

The starter tests omitted all trade-in logic, validation failures, flagship pricing, warranty discounts, express fees and stock delays. The initial branch-mode report showed 38% combined coverage: (18 + 8) / (40 + 28). Its branch-only coverage was 28.57%, with eight partially covered decisions.

Coverage interpretation

Statement coverage establishes that each executable statement ran. Branch coverage checks both outgoing paths of each decision. Neither establishes every possible execution path or proves correctness. Expected outputs and exceptions are asserted explicitly, and additional combinations test short-circuit logic.

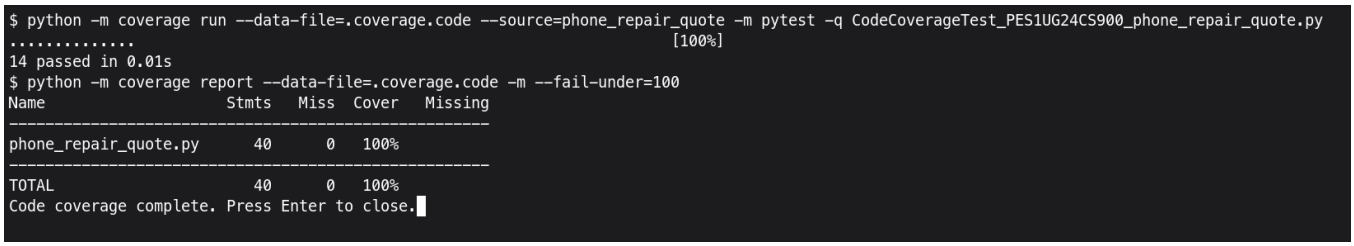
Test design

The branch suite checks all 32 part/tier/warranty/express combinations, seven invalid quote cases, and 24 stock/part/technician combinations. Trade-in tests cover ages 0, 1, 11, 12, 13, 23, 24, 25, 47, 48, 49 and 60 with intact and cracked screens. Eighteen rounding cases check prices 1, 101 and 9999 across all brackets. Ten validation cases cover zero/negative prices and negative ages, including validation before the old-phone early return.

The warranty AND condition is tested with warranty false, warranty true for screens, and warranty true for non-screens. The stock-delay OR condition is tested with screen, camera and neither. The input-validation OR condition is tested with invalid prices, valid prices and invalid ages, and both valid. The final branch report has zero missing statements and zero partial branches.

Code coverage: terminal evidence

Final results captured from the terminal after reaching 100% coverage.

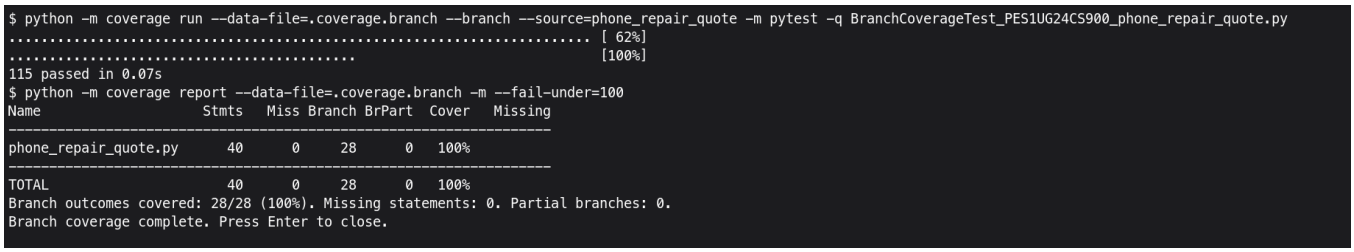


Commands and output

```
$ python -m coverage run --data-file=.coverage.code --source=phone_repair_quote -m pytest -q
CodeCoverageTest_PES1UG24CS900_phone_repair_quote.py
.............. [100%]
14 passed in 0.01s
$ python -m coverage report --data-file=.coverage.code -m --fail-under=100
Name                     Stmts   Miss  Cover   Missing
------------------------------------------------------
phone_repair_quote.py       40      0   100%
------------------------------------------------------
TOTAL                       40      0   100%
```

Branch coverage: terminal evidence

Final results captured from the terminal after reaching 100% coverage.



Commands and output

```
$ python -m coverage run --data-file=.coverage.branch --branch --source=phone_repair_quote -m pytest -q
BranchCoverageTest_PES1UG24CS900_phone_repair_quote.py
.......................................................................... [ 62%]
........................................                                 [100%]
115 passed in 0.07s
$ python -m coverage report --data-file=.coverage.branch -m --fail-under=100
Name                     Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------
phone_repair_quote.py       40      0     28      0   100%
-------------------------------------------------------------------
TOTAL                       40      0     28      0   100%
Branch outcomes covered: 28/28 (100%). Missing statements: 0. Partial branches: 0.
```

CodeCoverageTest: complete test file

CodeCoverageTest_PES1UG24CS900_phone_repair_quote.py

```
"""Statement coverage tests for the phone repair application."""

import pytest

from phone_repair_quote import repair_days, repair_quote, trade_in_value


@pytest.mark.parametrize(
    "part,tier,warranty,express,expected",
    [
        ("screen", "budget", False, False, 4600),
        ("battery", "flagship", True, True, 800),
        ("camera", "flagship", False, False, 5300),
        ("charging_port", "budget", False, False, 1100),
    ],
)
def test_repair_prices(part, tier, warranty, express, expected):
    assert repair_quote(part, tier, warranty, express) == expected


@pytest.mark.parametrize(
    "part,tier,message",
    [
        ("speaker", "budget", "unknown part"),
        ("battery", "premium", "unknown brand tier"),
    ],
)
def test_invalid_repair(part, tier, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        repair_quote(part, tier, False, False)


@pytest.mark.parametrize(
    "part,in_stock,technicians,expected",
    [
        ("battery", True, 2, 1),
        ("screen", False, 0, 8),
        ("charging_port", False, 1, 3),
    ],
)
def test_turnaround(part, in_stock, technicians, expected):
    assert repair_days(part, in_stock, technicians) == expected


@pytest.mark.parametrize(
    "price,age,cracked,expected",
    [
        (10000, 12, False, 6000),
        (10000, 24, False, 4000),
        (10000, 48, True, 1000),
        (10000, 49, False, 0),
    ],
)
def test_trade_in_prices(price, age, cracked, expected):
    assert trade_in_value(price, age, cracked) == expected


def test_invalid_trade_in_price():
    with pytest.raises(ValueError, match="^invalid input$"):
        trade_in_value(0, 12, False)
```

BranchCoverageTest: complete test file

BranchCoverageTest_PES1UG24CS900_phone_repair_quote.py

```
"""Decision outcomes, short-circuit cases, and boundaries for phone repairs."""

import pytest

from phone_repair_quote import repair_days, repair_quote, trade_in_value


@pytest.mark.parametrize("express,index", [(False, 0), (True, 1)])
@pytest.mark.parametrize(
    "part,tier,warranty,totals",
    [
        ("screen", "budget", False, (4600, 5100)),
        ("screen", "budget", True, (4600, 5100)),
        ("screen", "flagship", False, (8600, 9100)),
        ("screen", "flagship", True, (8600, 9100)),
        ("battery", "budget", False, (1800, 2300)),
        ("battery", "budget", True, (300, 800)),
        ("battery", "flagship", False, (3300, 3800)),
        ("battery", "flagship", True, (300, 800)),
        ("camera", "budget", False, (2800, 3300)),
        ("camera", "budget", True, (300, 800)),
        ("camera", "flagship", False, (5300, 5800)),
        ("camera", "flagship", True, (300, 800)),
        ("charging_port", "budget", False, (1100, 1600)),
        ("charging_port", "budget", True, (300, 800)),
        ("charging_port", "flagship", False, (1900, 2400)),
        ("charging_port", "flagship", True, (300, 800)),
    ],
)
def test_quote_combinations(part, tier, warranty, totals, express, index):
    assert repair_quote(part, tier, warranty, express) == totals[index]


@pytest.mark.parametrize(
    "part,tier,message",
    [
        ("speaker", "budget", "unknown part"),
        ("", "budget", "unknown part"),
        ("Screen", "budget", "unknown part"),
        ("battery", "premium", "unknown brand tier"),
        ("battery", "", "unknown brand tier"),
        ("battery", "Flagship", "unknown brand tier"),
        ("speaker", "premium", "unknown part"),
    ],
)
def test_quote_validation_and_precedence(part, tier, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        repair_quote(part, tier, False, False)


@pytest.mark.parametrize("technicians,index", [(0, 0), (1, 1), (2, 1)])
@pytest.mark.parametrize(
    "part,in_stock,days",
    [
        ("screen", True, (3, 1)),
        ("camera", True, (3, 1)),
        ("battery", True, (3, 1)),
        ("charging_port", True, (3, 1)),
        ("screen", False, (8, 6)),
        ("camera", False, (8, 6)),
        ("battery", False, (5, 3)),
        ("charging_port", False, (5, 3)),
    ],
)
def test_turnaround_combinations(part, in_stock, days, technicians, index):
    assert repair_days(part, in_stock, technicians) == days[index]
```

```
@pytest.mark.parametrize("cracked,index", [(False, 0), (True, 1)])
@pytest.mark.parametrize(
    "age,values",
    [
        (0, (6000, 3000)),
        (1, (6000, 3000)),
        (11, (6000, 3000)),
        (12, (6000, 3000)),
        (13, (4000, 2000)),
        (23, (4000, 2000)),
        (24, (4000, 2000)),
        (25, (2000, 1000)),
        (47, (2000, 1000)),
        (48, (2000, 1000)),
        (49, (0, 0)),
        (60, (0, 0)),
    ],
)
def test_trade_in_boundaries(age, values, cracked, index):
    assert trade_in_value(10000, age, cracked) == values[index]


@pytest.mark.parametrize("cracked,index", [(False, 0), (True, 1)])
@pytest.mark.parametrize(
    "price,age,values",
    [
        (1, 0, (0, 0)),
        (1, 13, (0, 0)),
        (1, 25, (0, 0)),
        (101, 0, (60, 30)),
        (101, 13, (40, 20)),
        (101, 25, (20, 10)),
        (9999, 0, (5999, 2999)),
        (9999, 13, (3999, 1999)),
        (9999, 25, (1999, 999)),
    ],
)
def test_trade_in_integer_rounding(price, age, values, cracked, index):
    assert trade_in_value(price, age, cracked) == values[index]


@pytest.mark.parametrize("cracked", [False, True])
@pytest.mark.parametrize(
    "price,age",
    [(0, 0), (-1, 12), (10000, -1), (0, -1), (0, 49)],
)
def test_trade_in_invalid_inputs(price, age, cracked):
    # Includes each side of the validation OR and validation before early return.
    with pytest.raises(ValueError, match="^invalid input$"):
        trade_in_value(price, age, cracked)
```

Reproducing the results

From the LAB6 folder, create and activate a virtual environment and install the pinned requirements. pytest.ini discovers both assignment test filenames as well as the original starter tests.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m coverage run --branch --source=phone_repair_quote -m pytest -q
python -m coverage report -m --fail-under=100
```

Separate suite commands are shown with their terminal evidence above. The --fail-under=100 option causes verification to fail if the measured application coverage falls below 100%.

Deliverables: separate code and branch test files, two terminal screenshots, coverage analysis, this report, and separate commits for code and branch coverage.